ARCA: analista de riesgo de crédito al consumo

Informe técnico

Guzmán Ignacio Pérez Ibarz, Jaime Pedrosa Comino e Ignacio Viadero Canduela
Modelos Generativos Profundos · Máster en IA · ICAI

2 de noviembre de 2026

1. El problema y el usuario

El equipo lo formamos Guzmán Ignacio Pérez Ibarz, Jaime Pedrosa Comino e Ignacio Viadero Canduela y ARCA es nuestro trabajo para la práctica de Modelos Generativos Profundos. ARCA es un agente para alguien que analiza riesgo en préstamos al consumo. Hoy esa persona hace tres cosas a mano. Primero busca el Euribor. Luego calcula en una hoja de cálculo la cuota el coste total la TAE y el ratio de endeudamiento de la operación. Y cuando duda de un plazo o de un límite se lee la Ley 16/2011 de crédito al consumo o la Directiva (UE) 2023/2225 que son textos bastante largos. En todos esos pasos se pueden cometer errores sobre todo con la TAE porque no tiene una fórmula directa y hay que resolverla por aproximaciones. Lo que hace el agente es juntar las tres cosas en una sola petición y dejar los cálculos a una herramienta en vez de al modelo. La idea de fondo es que los modelos pequeños se expresan bien pero calculan mal así que es mejor que no hagan las cuentas ellos.

Este es un ejemplo real del banco de evaluación con el cerebro `thinking`. El Euribor va fijo para que se pueda repetir.

```
Usuario: Quiero 12000 euros a 36 meses a Euribor a 12 meses mas 3 puntos.
         Cual es la cuota mensual?
1 get_euribor   {"tenor": "12M"}  ->  {"rate_pct": 2.5}
2 loan_metrics  {"principal": 12000, "annual_rate_pct": 5.5, "months": 36,
                 "other_monthly_debts": 0}
                ->  {"monthly_payment": 362.35, "total_cost": 1044.6,
                     "apr_pct": 5.64, "debt_ratio_pct": null}
3 final_answer  "362.35"      (3 pasos, 7463 tokens, correcta)
```

2. Arquitectura

El proyecto tiene cuatro fases que se van apoyando una en otra y una API por encima que las junta. La fase 1 enseña a un modelo pequeño a razonar paso a paso. La 2 le da herramientas. La 3 le da acceso a la ley. Y la 4 lo reúne todo en un agente que decide por sí mismo qué usar.

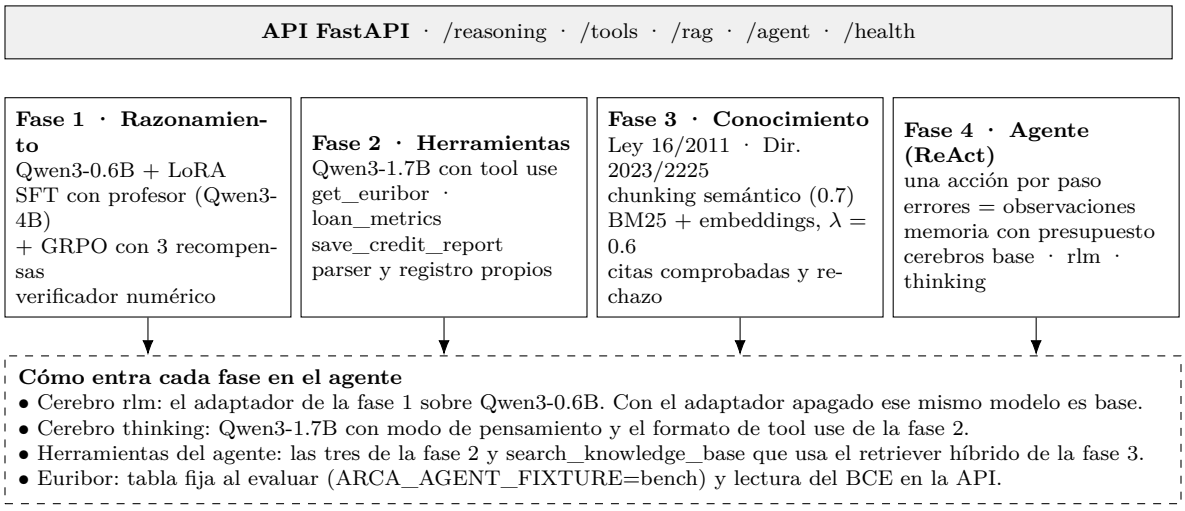


Figura 1: Las cuatro fases y cómo se conectan. Los modelos se cargan la primera vez que se usan.

Elegimos Qwen3 porque toda la familia comparte tokenizador y cabe en la GPU de 16 GB. El 0.6B es para entrenar porque es el que aguanta en memoria y el 1.7B para herramientas RAG y agente. Dentro del agente solo usamos la parte de recuperar del RAG porque la respuesta la escribe el cerebro del propio agente y no tiene sentido que escriban dos modelos. Descartamos la guía de la EBA porque había que bajarla a mano y el material del Banco de España porque venia con restos de la web pegados al texto. Tampoco resumimos la memoria del agente con el modelo por defecto porque cuesta una llamada extra cada vez que se comprime. Hay una variante con `ARCA_AGENT_SUMMARY=llm` para quien quiera probarla.

3. Fase 1: razonamiento

Datos, verificador y destilación

Los problemas los genera un script (`rlm/data/generar_dataset_credito.py`) y son de cuatro tipos: cuota con amortización francesa, TAE, coste total del crédito y ratio de endeudamiento. Cada enunciado trae ingresos deudas importe plazo tipo nominal y comisión de apertura. Pide el resultado con dos decimales dentro de `<answer>` para poder corregirlo con un programa. El test son 90 problemas que el modelo no ha visto: 19 de coste total, 23 de cuota, 21 de TAE y 27 de ratio. El verificador coge el número de `<answer>` y lo compara con el esperado con tolerancia 0 así que tiene que coincidir exactamente. Es un verificador muy estricto y tiene una limitación importante: dice si el número está bien pero no si el razonamiento está bien. Para entrenar hay 510 problemas (381 sin TAE y 129 de TAE) y GRPO usó 254.

Para el SFT necesitábamos ejemplos de razonamiento bien hecho y no los teníamos. Así que usamos un modelo más grande como profesor (Qwen3-4B) que resuelve los problemas y nos quedamos solo con sus trazas correctas según el verificador. Eso es destilar. En coste total y ratio salen trazas válidas. En **cuota (0 de 508)** y en **TAE (0 de 32)** no sale ninguna porque el profesor se queda sin tokens antes de cerrar la respuesta. Esto fue un problema importante desde el principio y condiciona todo lo que viene después. El fichero bruto de trazas tiene 1524 líneas y el final con el que entrenamos el SFT tiene 376.

SFT y GRPO

El SFT es LoRA sobre Qwen3-0.6B. LoRA es una forma de entrenar solo unas matrices pequeñas que se pegan al modelo en vez de tocar todos sus pesos y por eso cabe en nuestra GPU. Empezamos con rango 8 porque la tarea es muy estrecha y hay pocos datos. Usamos 3 épocas con tasa de aprendizaje 2e-4 longitud máxima 1024 y lote efectivo de 16. GRPO es un método de aprendizaje por refuerzo donde el modelo genera varias respuestas al mismo problema y se queda con las que sacan mejor recompensa que la media del grupo. Arranca del adaptador del SFT. Hicimos 100 pasos con tasa 2e-5 y acumulación de gradiente 2. Tuvimos que usar micro-lotes porque con el lote inicial no cabía en memoria. Cada paso tardó unos 102 s así que fueron 2 h 50 min en total.

Las recompensas son tres con pesos 1.0, 2.0 y 0.5. La primera es de formato (que `<think>` y `<answer>` estén

bien cerrados). La segunda es de exactitud y es la que más pesa porque usa el verificador. La tercera es de dominio y premia que el modelo derive la respuesta en vez de inventársela (al menos tres números distintos dentro del `<think>`) y que la respuesta salga limpia con dos decimales y sin símbolos. La elegimos porque lo peor que puede hacer un modelo en este dominio es dar una cifra que parece buena sin haberla calculado. El máximo que se puede sacar es 3.5.

Resultados

Evaluamos los tres modelos igual: decodificación voraz y 1024 tokens como máximo. GRPO casi duplica al SFT (23 aciertos frente a 13 de 90). La mejora viene de dos tipos. Coste total sube de 13 a 16 de 19 y ratio pasa de 0 a 7 de 27. **Cuota y TAE se quedan en 0 con los tres modelos.**

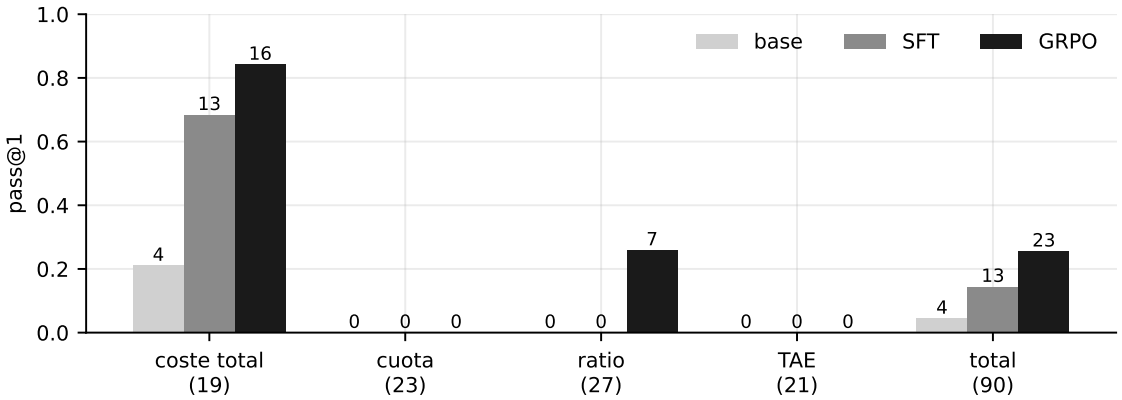


Figura 2: Aciertos por tipo en el test (el número sobre la barra y entre paréntesis los problemas de ese tipo). pass@1 total: base 0.044 · SFT 0.144 · GRPO 0.256.

	base	SFT	GRPO
Formato válido	0.31	0.21	0.41
Tokens medios generados	929	986	904

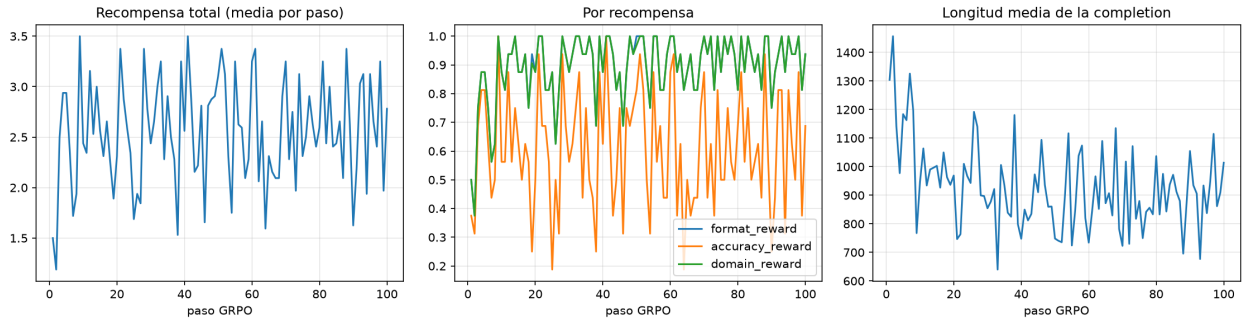


Figura 3: Curvas del entrenamiento de GRPO (100 pasos): recompensa total, cada una de las tres recompensas y longitud media de lo que genera. Se ve que el formato y el dominio suben y se pegan al 1 mientras que la exactitud sigue dando saltos.

Las curvas. Cada paso por separado es muy ruidoso como se ve en la figura así que comparamos la media de los diez primeros pasos con la de los diez últimos. La recompensa total sube de 2.30 a 2.65 (el máximo es 3.5). El formato sube de 0.72 a 0.93 y la recompensa de dominio de 0.72 a 0.93. **Pero la exactitud casi no se mueve: 0.61 a 0.63.** La longitud media baja de 1146 a 914 tokens y la desviación de la recompensa dentro de cada grupo de 1.37 a 1.07. Lo que aprendió GRPO sobre todo es a cerrar bien la respuesta y a pensar más corto. Es lo que más encaja con que en el test acierte 10 problemas más que el SFT: tiene más respuestas completas que se pueden corregir. No lo hemos separado con un experimento así que es lo que creemos y no algo comprobado. La recompensa de dominio es casi igual a la de formato (coinciden en 98

de los 100 pasos) porque lo que pide se cumple casi siempre cuando el formato es válido. Aporta poca señal propia y reconocemos que es un fallo de diseño nuestro. Ademas el 59 % de las respuestas de GRPO en el test sigue sin formato válido. En entrenamiento el formato llega a 0.93 y en test solo a 0.41. El test incluye cuota y TAE que el modelo no resuelve y en los que se corta y el entrenamiento solo tenía coste total y ratio así que no comparamos las dos cifras directamente.

Por qué cuota y TAE no mejoran. No entraron en GRPO. Los 254 problemas de entrenamiento son solo de coste total (131) y de ratio de endeudamiento (123). GRPO ha mejorado justo los dos tipos que entrenó (coste total de 13 a 16 y ratio de 0 a 7) y los otros dos se quedan donde los dejó el SFT. Aunque hubieran entrado el SFT no acertaba ninguno así que ningún grupo tendría respuestas correctas. En GRPO eso da ventaja cero y no hay gradiente que venga de la exactitud. Solo quedan las recompensas de formato y de dominio y esas enseñan a cerrar bien la respuesta pero no a calcular. Nuestra idea es que GRPO mejora lo que el modelo ya resuelve a veces pero no descubre lo que no sabe hacer nunca. Eso último es una interpretación y no lo hemos probado con un experimento aparte.

Cinco fallos

Revisamos a mano varios fallos del GRPO en el test y estos cinco nos parecieron los más representativos:

- **Se queda sin tokens en una cuenta larga (ratio).** Plantea bien el ratio y suma bien las deudas (408.35 + 1299.67 = 1708.02) pero se acaba el límite dividiendo entre 3722.57. Esperado 45.88.
- **Plantea bien pero falla la aritmética (coste total).** Multiplica 7.01 por 48 y le sale 336.96 en vez de 336.48. Da 1852.33 y era 1851.85. El plan incluía bien la comisión de apertura.
- **Se inventa la fórmula (cuota).** En un préstamo de 17694.46 € a 48 meses dice que la cuota es «(préstamo + comisión) entre meses más intereses» y da 370.43 en vez de 448.27.
- **No sabe qué es lo que calcula (TAE).** Duda si la TAE es el propio tipo nominal o si hay que sumar la comisión y no llega a hacer ninguna cuenta. La TAE necesita resolver una ecuación de forma iterativa y eso no se puede hacer a mano en 1024 tokens.
- **Confunde el concepto (cuota).** Lee «sistema de amortización francés» como «German amortization» y razona con ese nombre hasta que se trunca.

Hay otro fallo casi correcto (ratio 22.76 frente a 22.78) que la comprobación estricta a dos decimales da por malo. Es algo injusto pero preferimos un verificador estricto a uno que deje pasar cualquier cosa.

El paso de GRPO a mano

En `rlm/grpo_step.py` hicimos un paso del algoritmo y los 6 tests de `tests/test_grpo_step.py` pasan. La ventaja de cada respuesta es su recompensa menos la media del grupo dividida entre la desviación. Por eso si todas las respuestas de un grupo sacan lo mismo la ventaja es cero y ese grupo no enseña nada. Es lo que pasaría con cuota y TAE si entraran en GRPO con el SFT en cero, y es lo que pasó en los dos pasos donde las 8 respuestas sacaron el máximo (el 9 y el 41). El ratio entre la política nueva y la vieja se recorta para que un solo paso no mueva demasiado el modelo y la penalización KL lo ata a la referencia (en nuestra ejecución estaba desactivada porque `-beta` vale 0 por defecto).

4. Fase 2: herramientas

La idea de esta fase es que el modelo no haga las cuentas y llame a una función que las haga bien. Hay tres herramientas que son las que necesita el analista. `get_euribor` lee el BCE y cambia cada mes. `loan_metrics` da cuota, coste total, TAE y ratio con una sola fórmula y cuenta la comisión de apertura. `save_credit_report` es la única con efectos y pide confirmación. Las hicimos con un registro que convierte funciones tipadas en definiciones JSON y un parser propio de `<tool_call>`. El modelo es Qwen3-1.7B.

Primero medimos en un banco de 33 casos si elige bien la herramienta y los argumentos en la primera llamada. El prompt inicial daba selección 0.939 y argumentos 0.909 y con dos iteraciones llegamos a 1.000 y 0.970. Quitar los ejemplos no empeoró nada (1.0 y 1.0) y quitar las reglas lo empeoró poco (0.970 y 0.970). Como ese banco es el que usamos para iterar el prompt la cifra es de desarrollo y sale optimista. La cifra más honesta sale de 40 casos nuevos (10 por tipo):

Acierto	cuota	coste total	ratio	TAE	total
Sin herramientas	0	0.40	0	0	0.10 (4/40)
Con herramientas	1.00	0.80	0.50	0.40	**0.68 (27/40)**

Dónde falla. Cuando el modelo llama a la herramienta acierta siempre (27 de 27). Los 13 fallos tienen algo en común: no llama a nada. Dice lo que va a hacer o razona en el texto y la lista de llamadas queda vacía. Con `curl` a `/tools` vimos además un caso donde se salta el argumento opcional `upfront_fee` y devuelve una TAE de 10.36 % en vez de 12.60 %. Es un fallo sencillo pero en un banco sería grave porque el número parece razonable.

5. Fase 3: conocimiento

Corpus y gold set. Usamos la Ley 16/2011 (110000 caracteres) y la Directiva 2023/2225 (249000) bajadas con `rag.fetch_corpus`. Salen unos 500 fragmentos. El gold set son 50 preguntas con una cita literal del texto como evidencia. Las sacamos con ayuda de un modelo y las repasamos a mano. Se parecen bastante al texto (comparten un 0.55 de palabras) ninguna necesita dos fragmentos y alguna no se entiende sin contexto. Hay que tenerlo en cuenta al leer los resultados porque favorece a los métodos que buscan por palabras.

Chunking. Chunking es partir el texto en trozos para buscar en ellos. El chunking semántico corta donde cambia el tema y para eso mide la distancia entre frases seguidas con embeddings. Con Qwen3-Embedding-0.6B esas distancias son altas y cambian de un documento a otro (media 0.596 en la Ley y 0.497 en la Directiva). En las figuras de abajo se ve que casi todas las fronteras quedan por encima de las rayas rojas de 0.2 0.3 y 0.4. Los umbrales típicos de 0.2 a 0.4 cortaban entre el 73 % y el 98 % de las fronteras o sea casi cada frase y el texto quedaba troceado en frases sueltas. Por eso probamos un umbral fijo de 0.7 y los percentiles 70 y 80 y lo comparamos con el chunking recursivo de 800 caracteres.

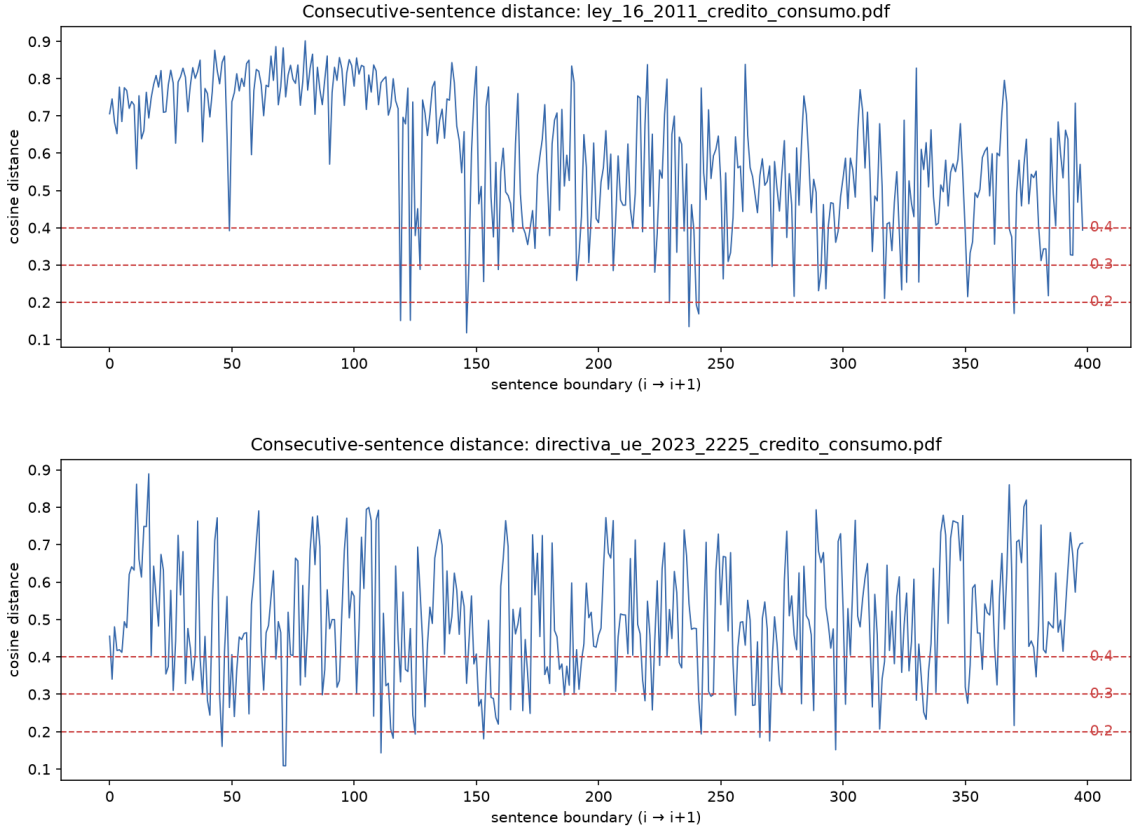


Figura 4: Distancia coseno entre frases consecutivas en las primeras 400 fronteras de la Ley 16/2011 (arriba) y de la Directiva 2023/2225 (abajo). Las rayas rojas son los umbrales de 0.2 0.3 y 0.4 que probamos al principio. En la Ley casi todo está por encima de 0.6 al comienzo y en la Directiva la nube se mueve entre 0.3 y 0.8.

Recall@5 / MRR por retriever:

Chunking	chunks	BM25	denso	híbrido (mejor λ)
recursive-800	543	0.65 / 0.42	0.53 / 0.43	0.65 / 0.50 (0.7)
semantic-0.7	498	0.68 / 0.55	0.66 / 0.50	**0.74 / 0.63** (0.6)
semantic-p70	605	0.66 / 0.53	0.62 / 0.40	0.74 / 0.58 (0.5)
semantic-p80	496	0.66 / 0.55	0.59 / 0.47	0.72 / 0.61 (0.5)

El híbrido mezcla la búsqueda por palabras (BM25) con la de embeddings usando un peso λ. Gana o empata en los cuatro y BM25 le gana al denso. Tiene sentido porque nuestras preguntas comparten muchas palabras con el texto pero no lo tomamos como regla general. El chunking semántico sale mejor (0.74 frente a 0.65) pero con 50 preguntas cada una pesa 0.02 y el error ronda ± 0.06. Además los chunks semánticos son más largos (hasta 1500 caracteres frente a 799) y λ se elige con las mismas preguntas con las que evaluamos. En la API dejamos semantic-0.7 híbrido y $\lambda = 0.6$.

Generación, citas y rechazo. El generador (Qwen3-1.7B con $k = 5$) cita fragmentos por identificador y comprobamos que cada cita esté entre los fragmentos recuperados. Las citas inventadas son **0 %**. Se abstiene en 8 de 8 preguntas sin respuesta en el corpus con un umbral de similitud de 0.569. Ese umbral se calibró con esas mismas 8 y el margen es mínimo (0.003). Pero es demasiado cauto: se niega en el 32 % de las preguntas que sí tienen respuesta (38 % con chunking recursivo) y más de la mitad de esas veces tenía delante el fragmento correcto. Que no invente citas no significa que sea fiel al texto. A «¿Qué conceptos forman parte del coste total del crédito?» da una lista incorrecta con citas que existen pero no la respaldan. Medir la fidelidad necesitaría un juez que no hemos hecho. Solo el 32 % de las respuestas cita el fragmento de la evidencia aunque esa métrica se queda corta porque solo da por bueno un fragmento.

6. Fase 4: agente

El bucle. Es un ReAct con una acción por paso: el modelo piensa elige una herramienta ve el resultado y repite hasta contestar. Si el modelo escribe varias llamadas solo se ejecuta la primera. Todo lo que falla es una observación y no una excepción: JSON mal formado, herramienta que no existe, tiempo máximo de 20 s o confirmación denegada. Así el modelo ve su propio error y puede corregirlo en vez de que el programa se caiga. Si repite la misma llamada la segunda no se ejecuta y a la tercera la tarea termina como fallida. La memoria deja fija la tarea original, recorta las observaciones largas y resume por extracción solo cuando se pasa del presupuesto de tokens. `save_credit_report` pide confirmación. En `/agent`, que es una sola llamada, la petición del usuario cuenta como confirmación y con `ARCA_AGENT_CONFIRM=deny` se bloquea.

El banco. Son 23 tareas. 21 encadenan dos o tres herramientas (Euribor, base de conocimiento, `loan_metrics` y `save_credit_report`) y 2 son imposibles (el Euribor de 2030 y el saldo de un cliente) y solo se aciertan si el agente dice que no se puede. La comprobación es automática con tolerancia de 0.02 y los esperados se recalculan en un test con la misma herramienta. Además del éxito medimos qué fracción de tareas usa de verdad las herramientas que hacían falta.

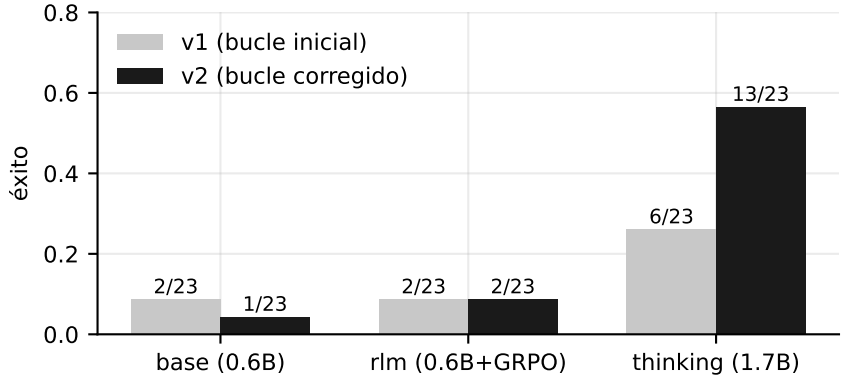


Figura 5: Tareas resueltas de 23 por cerebro antes y después de corregir tres fallos de nuestro bucle.

Cerebro	éxito v1 → v2	herram. requeridas v1 → v2	pasos v2	tokens v2
base (0.6B)	0.09 → 0.04	0.09 → 0.09	7.3	17625
rlm (0.6B + GRPO)	0.09 → 0.09	0.13 → 0.13	5.6	13319
thinking (1.7B)	0.26 → **0.57**	0.26 → **0.70**	2.8	7506

Tres fallos que eran nuestros. La primera ejecución dio 0.26 con el cerebro de 1.7B que en la fase 2 elegía bien las herramientas. Eso no cuadraba así que revisamos lo que escribía el modelo y vimos tres cosas. Primero imitaba la transcripción ReAct y escribía él mismo el `Observation:` con un valor inventado y seguía con un `final_answer` basado en ese valor. Ahora se descarta todo lo que viene después de un `Observation:` escrito por el modelo. Segundo escribía sumas dentro del JSON (`"annual_rate_pct": 2.5 + 3`). Esa llamada no era válida y el bucle saltaba a la siguiente que sí lo era, que justo era el `final_answer` inventado. Ahora la suma se calcula y si la primera llamada sigue mal no se ejecuta ninguna de las siguientes. Tercero escribía `Action: {json}` sin las etiquetas y lo tratábamos como texto. Ahora se acepta. Con eso `thinking` pasa de 6 a 13 tareas de 23. Estos arreglos los hicimos mirando tareas de este mismo banco así que el 0.57 es de desarrollo y seguramente sale optimista.

Los 10 fallos que quedan en `thinking` son del modelo. Suma mal el diferencial («más 5 puntos» sobre un 2.0 lo escribe como 2.05). En las tareas que mezclan la ley con un cálculo a veces busca con `top_k` = 1 y recibe el apartado del 0.5 % (menos de un año) cuando se pedía el de más de un año. Otras veces contesta solo el dato legal sin llamar a `loan_metrics` o se inventa el porcentaje. En el ratio no pasa `monthly_income`, la herramienta devuelve nulo y lo calcula a mano sin la cuota (13.64 en vez de 27.79). Y en la tarea imposible del Euribor de 2030 da la tasa actual con una frase de cautela en vez de decir que no se puede saber.

Los cerebros de 0.6B no fallan por el bucle. En `base` 12 de 22 fallos no llaman a ninguna herramienta y muchos repiten `Action: loan_metrics` hasta gastar los 10 pasos. `rlm` se inventa cifras redondas como 1000.00. El adaptador de la fase 1 se entrenó para `<think>` y `<answer>` y no para llamar herramientas así que no se transfiere. Como `rlm` y `thinking` también se diferencian en el tamaño no podemos atribuir la diferencia solo al entrenamiento ni solo al tamaño.

Agente JSON frente a agente de código. Quisimos comparar nuestro agente que escribe una llamada JSON por paso con uno que escribe código Python (`CodeAgent` de smolagents) con el mismo cerebro las mismas herramientas y el mismo banco en 8 tareas:

Agente (8 tareas)	éxito	herramientas requeridas	pasos	tokens
JSON (nuestro bucle)	0.62 (5/8)	0.88 (7/8)	3.0	8091
Código (smolagents)	0.38 (3/8)	0.50 (4/8)	4.4	32744

Con el mismo modelo el agente JSON acierta más y gasta unas cuatro veces menos tokens. Son solo 8 tareas así que una tarea más o menos cambia 0.12 y lo tomamos como una tendencia y no como una medida firme. Creemos que el motivo es el formato. El agente de código necesita que el modelo cierre un bloque `<code>` y con el modo de pensamiento el 1.7B gasta los tokens razonando y se corta antes, así que smolagents da error de parseo y reintenta hasta el límite de pasos (por eso gasta más tokens y más pasos). En el JSON la llamada es una línea corta y además la reparamos si viene mal escrita. De hecho la ejecución del agente de código tardó más de una hora por esos reintentos. Un fallo típico del agente de código es sumar 0.0003 al resultado de `get_euribor`, que es un diccionario (`{"rate_pct": ...}`), y llamar a `loan_metrics` con un argumento que no existe (`monthly_payment=0`).

7. Lo que no funcionó

- **El profesor no resolvía cuota ni TAE.** Sin trazas válidas de esos tipos el SFT no los aprende. Tampoco los metimos en GRPO y aunque lo hubiéramos hecho con el SFT en cero no habría grupos con aciertos y la ventaja sería cero. Asi que cuota y TAE siguen en 0 de 23 y 0 de 21.
- **Dos medidas distintas del SFT.** Una primera evaluación dio 0.278 (25 de 90) y con el protocolo común de 1024 tokens el SFT se queda en 0.144. No son comparables y usamos solo la segunda. La diferencia está sobre todo en el ratio (10 de 27 antes y 0 de 27 ahora) que es un cálculo largo y probablemente se trunca con el protocolo actual.
- **Casi toda la mejora durante GRPO fue de formato.** La exactitud en entrenamiento se quedó plana y la recompensa de dominio resultó ser casi una copia de la de formato (98 de 100 pasos iguales). Con otra recompensa de dominio que midiera algo distinto habría más señal.

- **El umbral por defecto del chunking semántico** (0.25) trituraba el texto en frases sueltas con estos embeddings. Por eso comparamos 0.7 y percentiles.
- **El bucle del agente** tenía tres fallos nuestros que escondían lo que el modelo sí sabía hacer (sección 6).
- **El RAG es demasiado cauto** y el umbral de rechazo apenas separa las 8 preguntas sin respuesta del resto (margen 0.003). No tocamos el prompt de abstención por falta de tiempo.
- **Los cerebros de 0.6B** no son agentes porque no hacen llamadas válidas de forma fiable y el adaptador de razonamiento no lo arregla.
- **El agente de código** se atasca con el modo de pensamiento y tardó más de una hora en 8 tareas.

8. Extensión proyecto final

Con lo que hemos visto hay bastantes cosas que nos gustaría hacer en el proyecto final con más tiempo y más cómputo. Las ordenamos más o menos por la mejora que esperamos de cada una:

- **Resolver cuota y TAE de verdad.** Es lo más importante porque ahora están en 0 de aciertos. Destilar con un profesor y un presupuesto de tokens que sí cierren esos tipos (o con trazas que usen una calculadora) y repetir GRPO con ejemplos que el modelo resuelva alguna vez para que haya ventaja en los grupos.
- **Rediseñar la recompensa de dominio.** Ahora es casi igual que la de formato y aporta poco. Habría que hacer una que mida algo distinto como que los pasos intermedios sean coherentes o que se use la fórmula correcta.
- **Evaluar con más tokens y varias semillas.** Medir pass@1 con 2048 tokens y varias ejecuciones para poner intervalos de confianza y repetir SFT frente a GRPO con y sin KL. Con 90 problemas y una sola ejecución no se distinguen bien las diferencias pequeñas.
- **SFT de uso de herramientas** para los cerebros pequeños y un banco de cadenas de herramientas más grande con una parte que no usemos para iterar. Así el cerebro de 0.6B podría llegar a funcionar como agente y la cifra de desarrollo deja de ser optimista.
- **RAG.** Un reranker, un juez de fidelidad, un gold set más grande escrito a mano con preguntas que necesiten dos fragmentos y un `top_k` mayor por defecto en la herramienta del agente. También habría que tocar el prompt de abstención para que no se niegue tanto y calibrar el umbral con más de 8 preguntas.
- **Validar los argumentos obligatorios** antes de ejecutar una herramienta. Por ejemplo avisar de que el ratio necesita `monthly_income` en vez de devolver nulo.
- **Repetir JSON frente a código** con más tareas y sin modo de pensamiento para separar cuánto es culpa del formato y cuánto del modelo.
- **Despliegue.** Dejar la API con una URL fija y un docker que se levante con un solo comando para no depender de la sesión de la DGX.

Anexos

Repositorio: `https://github.com/Guso12345678/dgm-arca` (el cuaderno completo está en `EXPERIMENTS.md`).
URL de la API durante la corrección: `https://nonswimming-angle-platonically.ngrok-free.dev` (la documentación está en `/docs`). Solo responde mientras la sesión de la DGX y el túnel estén abiertos.

Cómo reproducir (entorno: `uv sync -extra train -extra rag -extra agent` y tests con `uv run pytest`):

```
# Fase 1
uv run python -m rlm.evaluate --data rlm/data/rlm_data_test.jsonl --adapters base=none sft=rlm/weights/sft_lora
      grpo=rlm/weights/grpo_lora --max-new-tokens 1024
uv run python -m rlm.report_phase1 by-type # tambien: curves, failures
# Fase 2
uv run python -m tool_use.compare --per-type 10
# Fase 3
uv run python -m rag.evaluate --gold rag/gold/gold.jsonl --k 5 --calibrate rag/gold/unanswerable.jsonl --
     generation
# Fase 4
ARCA_AGENT_FIXTURE=bench uv run python -m agent.evaluate --bench agent/bench/tasks.jsonl --brains base rlm
     thinking
uv run python -m agent.compare_codeagent --brains thinking --limit 8
# API
ARCA_RLM_ADAPTER=rlm/weights/grpo_lora ARCA_RAG_CHUNKING=semantic ARCA_RAG_SEMANTIC_THRESHOLD=0.7
     ARCA_RAG_LAMBDA=0.6 ARCA_RAG_MIN_SIMILARITY=0.569 uv run arca-api
```